

Инвариантное задание 1.2. Разработать скрипт для автоматизации установки и настройки прикладного ПО.

1. Цель работы

Разработать скрипт автоматизации установки и настройки прикладного программного обеспечения, а также исследовать способы и инструменты создания резервных копий (образов) операционной системы.

2. Задачи

1. Изучить способы автоматизации установки ПО под Windows.
2. Разработать BAT-скрипт для установки 30 программ из предоставленного списка.
3. Провести тестирование скрипта в виртуальной среде.
4. Исследовать технологии создания образа ОС и резервного копирования.
5. Зафиксировать результаты и проблемы, возникшие в процессе работы.

3. Исследование способов автоматизации установки ПО

3.1. BAT-скрипты

BAT (batch) — текстовый файл с командами для интерпретатора cmd.exe.

Основные возможности:

- последовательное выполнение команд установки;
- запуск установщиков с ключами тихого режима;
- проверка прав администратора, логирование, обработка ошибок.

Преимущества: работает на любой Windows без дополнительных компонентов.

Ограничения: слабые средства диагностики, ограниченная работа с выводом дочерних процессов.

3.2. MSI и msixexec

MSI — стандартный формат установочных пакетов Windows Installer.

Установка выполняется утилитой msixexec.exe со следующими ключами:

Ключ	Назначение
/i	установить пакет
/x	удалить пакет
/qn	полностью тихий режим
/qb	тихий режим с прогресс-баром
/norestart	не перезагружать автоматически
/L*v log.txt	подробный лог установки

```
msiexec /i "package.msi" /qn /norestart /L*v "C:\Logs\install.log"
```

Преимущества: стандарт Microsoft, надёжный, встроен в систему.

Ограничения: не все программы распространяются в MSI-формате.

3.3. Chocolatey

Менеджер пакетов для Windows. Сам скачивает установщик и запускает его с нужными ключами.

```
choco install vscode -y
```

Преимущества: единый интерфейс для большинства программ, автоматическое разрешение зависимостей.

Ограничения: не для всех программ есть пакеты; часть пакетов ссылается на устаревшие или более недоступные URL дистрибутивов (подтверждено на практике, см. раздел 6).

3.4. Winget

Встроенный в Windows 10/11 менеджер пакетов от Microsoft. Аналог Chocolatey, но часть пакетов качает с CDN-зеркал производителей.

```
winget install --id Google.Chrome --exact --silent --accept-package-agreements --accept-source-agreements
```

Преимущества: встроен в Windows, для части пакетов есть отдельные CDN, не совпадающие с основным доменом производителя.

Ограничения: покрывает не всё ПО из списка; поведение зависит от версии клиента winget.

3.5. Сравнение подходов

Метод	Плюсы	Минусы
BAT	Универсален, не требует зависимостей	Слабая диагностика
MSI	Стандарт Microsoft, надёжен	Не все программы в MSI
Chocolatey	Единый интерфейс, автоустановка	Часть пакетов устарела/сломана
Winget	Встроен в Windows	Покрывает не всё ПО

Вывод: оптимальна комбинированная стратегия — Chocolatey для основного списка, winget и прямые ссылки (включая официальные MSI-дистрибутивы) для проблемных случаев.

4. Разработка скрипта

4.1. Структура

Скрипт `install_software.bat` состоит из шести этапов:

1. Проверка прав администратора — через `net session`.
2. Установка Chocolatey — если не установлен.
3. Установка основного списка ПО через Chocolatey (20 программ).
4. Установка ПО вне Chocolatey — KNIME, Arc, Yandex Browser, Google Chrome (`winget`); Yandex.Telemost (официальный MSI-инсталлятор, тихая установка); Sber Jazz, Anaconda3 (прямые ссылки).
5. Установка расширений Visual Studio Code — 9 расширений.
6. Включение компонентов WSL и регистрация автозапуска второго скрипта через `schtasks` (задача снимает сама себя после выполнения).

Дополнительный скрипт `install_wsl.bat` устанавливает ядро WSL 2, задаёт версию WSL по умолчанию и ставит обе требуемые версии Ubuntu (22.04 и 24.04) через `winget`.

4.2. Ключевые технические решения

Проверка прав администратора:

```
net session >nul 2>&1
if %errorLevel% neq 0 (
```

```

    echo [!] Требуется права администратора.
    pause
    exit /b 1
)

```

Корректная обработка кодов возврата Chocolatey (0 — успех, 3010/1641 — успех с требованием перезагрузки, остальное — ошибка):

```

call choco install %~1 -y --no-progress >> "%MAIN_LOG%" 2>&1
set "RC=!errorlevel!"
if "!RC!"=="0" (
    echo      [OK] %~2 установлен
) else if "!RC!"=="3010" (
    echo      [OK] %~2 установлен ^(требуется перезагрузка^)
) else if "!RC!"=="1641" (
    echo      [OK] %~2 установлен ^(инициирована перезагрузка^)
) else (
    echo      [ERR] Не удалось установить %~2 ^(код !RC!^)
)

```

Тихая установка Yandex.Telemost через официальный MSI — заменила ранний вариант с прямой ссылкой на .exe, который блокировал скрипт финальным окном инсталлятора (см. проблему 6.1):

```

msiexec /i "%TMP_DIR%\YTelemostSetup.msi" /qn ALLUSERS="1"
MSIINSTALLPERUSER="" ^
    SKIP_LAUNCH=1 NODESKTOPSHORTCUT=1 /norestart

```

SKIP_LAUNCH=1 не даёт программе запуститься после установки — тем самым устраняется сама причина зависания, а не её следствие.

Регистрация автозапуска install_wsl.bat через задачу планировщика с последующим самоудалением задачи (см. проблему 6.2):

```

schtasks /create /tn "ITiEO_InstallWSL" /tr "\"%~dp0install_wsl.bat\""" /sc
onlogon /rl highest /f
REM ...
REM в конце install_wsl.bat:
schtasks /delete /tn "ITiEO_InstallWSL" /f

```

Установка обеих требуемых версий Ubuntu в install_wsl.bat:

```

winget install --id Canonical.Ubuntu.2204 --exact --silent --accept-package-
agreements --accept-source-agreements
winget install --id Canonical.Ubuntu.2404 --exact --silent --accept-package-
agreements --accept-source-agreements

```

Логирование в папку скрипта и уборка временных файлов по завершении:

```

set "LOG_DIR=%~dp0"
set "TMP_DIR=%TEMP%\SoftwareInstall"
set "MAIN_LOG=%LOG_DIR%install_main.log"

```

```
REM ... в конце скрипта:  
rmdir /s /q "%TMP_DIR%" >nul 2>&1
```

4.3. Листинг (фрагмент)

```
@echo off  
chcp 866 >nul  
setlocal EnableDelayedExpansion  
title Автоматическая установка ПО  
  
REM --- Проверка прав администратора ---  
net session >nul 2>&1  
if %errorLevel% neq 0 (  
    echo [!] Требуется права администратора.  
    pause  
    exit /b 1  
)  
  
REM --- Этап 1: Chocolatey ---  
where choco >nul 2>&1  
if %errorLevel% neq 0 (  
    powershell -NoProfile -ExecutionPolicy Bypass -Command "..."  
)  
  
REM --- Этап 2: основной список через :install ---  
call :install vscode "Visual Studio Code"  
call :install docker-desktop "Docker Desktop"  
REM ... и т.д. (полный список – 20 пакетов)  
  
REM --- Этап 3: ПО вне Chocolatey ---  
REM KNIME/Arc/Yandex Browser/Chrome – winget; Telemost – msixexec /qn; Jazz,  
Anaconda – прямые ссылки  
  
REM --- Этап 4: расширения VS Code ---  
call :install_ext ms-python.python "Python"  
REM ... и т.д. (9 расширений)  
  
REM --- Этап 5: включение компонентов WSL 2 ---  
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-  
Linux /all /norestart  
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all  
/norestart  
  
REM --- Этап 6: регистрация автозапуска install_wsl.bat ---  
schtasks /create /tn "ITiEO_InstallWSL" /tr "\"%~dp0install_wsl.bat\"/" /sc  
onlogon /rl highest /f
```

Полный листинг обоих файлов — в приложении к отчёту.

5. Тестирование

5.1. Тестовая среда

- Виртуальная машина: Oracle VirtualBox, гостевая ОС Windows 10 22H2 (подтверждено путями в логах: пользователь C:\Users\ vboxuser\...).

- Ресурсы ВМ: ограничены (по ходу тестирования выявлена нехватка свободной памяти хоста — см. проблему 6.4);
- Способ установки: последовательный прогон `install_software.bat` из-под учётной записи с правами администратора.

5.2. Результаты установки

Программа	Способ	Результат	Примечание
Visual Studio Code	choco	Успешно	установлен без ошибок
Docker Desktop	choco	Ошибка	код 1603 (ERROR_PIPE_BUSY 231) → см. 6.4; повтор не выполнялся
PyCharm Community	choco	Ошибка	геоблокировка, HTTP 451 → см. 6.5
Git, Git (расширение)	choco	Успешно	
GitHub Desktop	choco	Успешно	
Maxima	choco	Успешно	
GIMP	choco	Ошибка*	таймаут 1200 сек при закатке → см. 6.6; штатно устанавливался в более ранних тестовых прогонах, повтор с исправленным таймаутом не выполнялся
Julia, Python, Rust	choco	Успешно	включая расширения VS Code
MSYS2 UCRT64	choco	Успешно	
Zettlr, MiKTeX, TeXStudio	choco	Успешно	
Far Manager, SumatraPDF	choco	Успешно	
Flameshot, Qalculate!, 7-Zip	choco	Успешно	
Firefox, Microsoft Edge	choco	Успешно	
KNIME Analytics Platform	winget	Ошибка	сетевой таймаут при закатке → см. 6.8
Arc Browser	winget	Ошибка	HTTP 403 при закатке → см. 6.9
Yandex Browser	winget	Успешно	

Программа	Способ	Результат	Примечание
Google Chrome	winget	Успешно	
Yandex.Telemost	MSI (msiexec)	Не подтверждено	GUI-режим прошёл без ошибок, но итог не подтверждён по реестру → см. 6.1
Sber Jazz	прямая ссылка	Ошибка	повреждённая/неполная загрузка → см. 6.8; повтор не выполнялся
Anaconda3	прямая ссылка	Ошибка	повреждённая/неполная загрузка (~500 МБ из ~1 ГБ) → см. 6.8; повтор не выполнялся
WSL 2 / Ubuntu 22.04 и 24.04	dism + winget	Успешно	обе версии подтверждены; задача автозапуска корректно удалила себя

Итог: из 30 позиций списка 22 установлены успешно, 1 (Yandex.Telemost) — с неподтверждённым статусом, 7 не установлены. Для всех 7 неудач причина диагностирована и задокументирована (раздел 6); все относятся к внешним факторам — геоблокировка, ограничения тестовой ВМ по ресурсам и сети, повреждение файла при загрузке, — а не к ошибкам логики самого скрипта. Часть неудачных пунктов не переустанавливалась повторно из-за ограниченного времени тестирования; при устранении причины (см. рекомендации в 6.x) ожидаемо устанавливаются штатно.

6. Проблемы и решения

6.1. Yandex.Telemost блокировал выполнение скрипта

Проблема: прямая установка Telemost из .exe-дистрибутива с ключом /S не открывала видимых окон в процессе установки, но после её завершения инсталлятор показывал финальное окно и сам запускал Telemost — cmd, ожидающий завершения процесса (start /wait), зависал до тех пор, пока это окно не закрывали.

Решение: переход на официальный MSI-дистрибутив Yandex.Telemost с ключами /qn (без любых окон) и SKIP_LAUNCH=1 (программа не

запускается после установки), согласно официальной документации Яндекс 360. Устраняет причину зависания, а не борется со следствием через принудительное завершение процесса.

Статус на момент отчёта: MSI-скрипт выполнен вручную без ключа /qn (в обычном режиме) и прошёл без видимых ошибок — зависание, характерное для .exe-версии, не повторилось. Однако факт успешной установки не подтверждён проверкой реестра (Get-ItemProperty ...Uninstall*), поэтому в таблице 5.2 статус отмечен как предположительный, а не окончательно подтверждённый.

6.2. Задача планировщика могла запускаться повторно

Проблема: install_software.bat регистрировал задачу schtasks /sc onlogon для запуска install_wsl.bat, но нигде не снимал её после выполнения — при каждом следующем входе в систему WSL-скрипт запускался бы заново.

Решение: в конец install_wsl.bat добавлена команда schtasks /delete /tn "ITiEO_InstallWSL" /f — задача удаляет саму себя сразу после выполнения.

6.3. Ограниченные ресурсы тестовой VM

Проблема: в ходе тестирования выявлена нехватка свободной оперативной памяти на хосте, что напрямую сказывается на скорости и стабильности установки внутри тестовой виртуальной машины (см. также проблему 6.5). Важная оговорка: это особенность именно тестового стенда — в реальных компьютерных аудиториях кафедры скрипт будет выполняться на полноценных физических машинах, а не в VM, поэтому часть рекомендаций ниже относится только к дальнейшему тестированию.

Рекомендации для дальнейшего тестирования (тестовая VM): выделять под VM больше памяти и ядер CPU; использовать фиксированный, а не динамический VDI-диск — это устраняет искажения, специфичные именно для виртуальной среды, и не относится к аудиториям.

Рекомендации для аудиторий кафедры (физические машины): объём памяти на реальном ПК фиксирован железом, поэтому вместо «выделения

ресурсов» стоит добавлять исключения в Windows Defender для папок установки на время развёртывания (актуально независимо от того, ВМ это или реальная машина) и по возможности запускать установку в часы, когда компьютер не занят фоновыми задачами. Если по итогам развёртывания на реальном железе всё равно будут всплывать похожие транзистные ошибки (тип 1603/231) — это будет говорить уже не о ресурсах ВМ, а о нагрузке конкретной машины в моменте, и решается тем же способом — повторным запуском установки.

6.4. Docker Desktop — ошибка 1603

Проблема: choco install docker-desktop завершился с кодом 1603 (Generic MSI Error). Повторный запуск с ключом подробного логирования (--install-arguments="/l*v C:\docker-desktop_msi_install.log") показал точную причину:

```
SFXCA: Failed to create communication pipe for new CA process. Error  
code: 231  
CustomAction ValidateInstallParameters returned actual error code 1603
```

Код 231 в Windows — ERROR_PIPE_BUSY («все экземпляры канала заняты»): встроенное действие MSI (ValidateInstallParameters) не смогло создать именованный канал для связи со своим дочерним процессом. Ошибка происходит на самом раннем этапе установки, до любых действий, связанных с Hyper-V/WSL-бэкендом Docker, — то есть причина не в конфигурации виртуализации, а в нехватке системных ресурсов ВМ в момент установки (см. 6.4).

Решение: повторная попытка установки на менее загруженной системе / с большим объёмом выделенной ВМ памяти; при необходимости — отключение Defender на время установки. Ошибка признана транзистной ошибкой окружения, не связанной с самим пакетом или скриптом.

6.5. PyCharm Community — геоблокировка JetBrains

Проблема: choco install pycharm-community завершился с кодом 404 (это код возврата внутреннего PowerShell-скрипта пакета), однако в теле лога видна настоящая причина:

```
Attempt to get headers for  
https://download.jetbrains.com/python/pycharm-2025.3.exe failed.  
"The remote server returned an error: (451) Unknown."
```

451 — это реальный HTTP-ответ сервера JetBrains («Unavailable For Legal Reasons»), то есть ограничение загрузки с IP-адресов РФ. Внутренний скрипт пакета Chocolatey перехватывает эту ошибку и завершается собственным кодом 404, из-за чего в консоли видна только вторичная цифра — настоящую причину нужно искать в логе.

Решение: повторная установка с включённым VPN, либо — для аудиторий кафедры — заранее подготовленный офлайн-дистрибутив PyCharm.

6.6. GIMP — таймаут закачки

Проблема: choco install gimp завершился ошибкой Chocolatey timed out waiting for the command to finish (лимит --execution-timeout=1200, то есть 20 минут). Оценка по наблюдаемой скорости канала (около 125 КБ/с) показывает, что дистрибутив GIMP (250+ МБ) объективно не укладывается в 20 минут — сам лимит, добавленный нами же для диагностики зависаний, оказался ниже, чем нужно для этого канала.

Решение: лимит увеличен до 3000 секунд (50 минут). В более ранних тестовых прогонах (без ограничения по времени) GIMP устанавливался штатно — то есть проблема именно в тайминге, а не в самом пакете. Повторная установка с новым лимитом на момент подготовки отчёта не выполнялась из-за ограниченного времени тестирования.

6.7. Anaconda3 и Sber Jazz — повреждённая загрузка

Проблема: оба пакета скачиваются напрямую через Invoke-WebRequest, без проверки контрольной суммы и без докачки (в отличие от Chocolatey/winget, где хэш установщика сверяется автоматически). Файл Anaconda3.exe запустился с ошибкой NSIS «Installer integrity check has failed» — при проверке размера выяснилось, что скачалось около 500 МБ из ожидаемого ~1 ГБ, то есть загрузка оборвалась на середине. Jazz.exe дал ту же ошибку.

Решение/рекомендация: повторное скачивание файлов (проблема транзистентная, той же природы, что и общая нестабильность сети на тестовой ВМ, см. 6.4); для финальной версии скрипта рекомендуется добавить простую проверку размера файла после скачивания, прежде чем запускать установку — например, отказ от установки, если размер файла меньше ожидаемого порога. Повторная загрузка на момент отчёта не выполнялась.

6.8. KNIME Analytics Platform — сетевой таймаут

Проблема: установка через winget завершилась ошибкой InternetReadFile() failed, код 0x80072ee2 (ERROR_INTERNET_TIMEOUT) при загрузке с download.knime.org.

Решение/рекомендация: повторная попытка на более стабильном канале; при системных повторениях — рассмотреть предварительную загрузку дистрибутива и раздачу с локального сервера кафедры, как и для других тяжёлых/нестабильных источников.

6.9. Arc Browser — HTTP 403 при загрузке

Проблема: winget install --id TheBrowserCompany.Arc успешно поставил зависимость (Windows App Runtime 1.5), но сам установочный файл Arc.x64.msix вернул Download request status is not success, 0x80190193 : Forbidden (403).

Возможная причина: разработчик Arc, компания The Browser Company, в мае 2025 года официально объявила о прекращении активной разработки браузера в пользу нового продукта Dia — для Arc с тех пор выпускаются только исправления безопасности. Это не доказывает напрямую причину именно 403-ответа, но не исключено, что инфраструктура раздачи файлов для отдельных регионов поддерживается уже не так надёжно, как раньше.

Решение/рекомендация: поскольку Arc фактически не развивается и жёсткой необходимости именно в нём нет, для аудиторий кафедры стоит рассмотреть исключение Arc из списка или замену на активно поддерживаемый браузер.

7. Резервное копирование и создание образа ОС

7.1. Инструменты

Инструмент	Назначение	Плюсы	Минусы
wbadmin	Полный образ системы	Встроен в Windows, простые команды	Устаревший интерфейс, ограниченная гибкость
dism /capture-image	Создание WIM-образа раздела	Стандарт Microsoft, гибкость	Требует WinPE или раздела восстановления
Точки восстановления	Снимок системных файлов и реестра	Быстро, встроено	Не полный образ диска
sysprep + WIM	Эталонный образ для клонирования	Стандарт массового развёртывания	Сложнее в настройке
Сторонние (Acronis, Clonezilla)	Полный образ диска	Удобный интерфейс, инкрементальные бэкапы	Платное ПО / отдельная загрузочная среда

7.2. Примеры команд

Полный бэкап критичных томов:

```
wbadmin start backup -backupTarget:D: -include:C: -allCritical -quiet
```

Создание WIM-образа:

```
dism /capture-image /imagefile:D:\backup\windows.wim /capturedir:C:\  
/name:"Windows_ITiEO" /compress:max
```

Развёртывание образа:

```
dism /apply-image /imagefile:D:\backup\windows.wim /index:1 /applydir:C:\
```

Точка восстановления:

```
Checkpoint-Computer -Description 'Backup_ITiEO' -RestorePointType  
'MODIFY_SETTINGS'
```

7.3. Рекомендация для аудиторий

Оптимальный подход для массового развёртывания в компьютерных аудиториях кафедры:

1. Создание эталонного образа на одной машине: установка Windows, всех драйверов и ПО; настройка параметров системы; подготовка к клонированию — sysprep /generalize /oobe /shutdown.

2. Захват образа: загрузка с WinPE, `dism /capture-image` → WIM-файл на внешний носитель.

3. Развёртывание на N машин: загрузка с WinPE, `dism /apply-image` — разворачивание за 10–15 минут на машину.

Преимущества: установка ПО выполняется один раз, экономия времени при развёртывании, гарантированная идентичность конфигураций.

8. Выводы

1. Разработан и протестирован BAT-скрипт для автоматической установки ПО из списка кафедры (30 позиций). При тестовом прогоне на виртуальной машине 22 программы установлены успешно, 1 (Yandex.Telemost) — с неподтверждённым до конца статусом, 7 не установлены; причина каждого сбоя диагностирована и задокументирована (раздел 6).

2. Оптимальной стратегией является комбинирование менеджеров пакетов (Chocolatey, winget) и официальных прямых ссылок/MSI-дистрибутивов: менеджеры дают единый интерфейс, автоматическое разрешение зависимостей и проверку хэша устанавливаемых файлов; прямые ссылки и MSI полезны там, где менеджеры не справляются, но требуют собственной проверки целостности загрузки, которой в текущей версии скрипта не хватает (см. 6.8).

3. Подтверждённые на практике проблемы автоматизации делятся на три группы: геоблокировка отдельных источников по IP (JetBrains — HTTP 451); ограничения тестовой ВМ по ресурсам и сети, влияющие на стабильность как долгих загрузок (GIMP, KNIME), так и самих MSI-инсталляций (Docker Desktop — ERROR_PIPE_BUSY); и особенности конкретных источников/пакетов, не связанные с автоматизацией напрямую (перебои с Arc, вероятно как следствие фактического прекращения разработки продукта).

4. Для практического применения в аудиториях целесообразно:
использовать эталонный образ системы (sysprep + dism) вместо последовательной установки на каждой машине — тогда перечисленные сетевые проблемы возникают один раз при подготовке эталона, а не на каждой машине аудитории; предварительно скачивать тяжёлые или нестабильно доступные дистрибутивы и распространять через локальный файловый сервер кафедры; добавить в скрипт проверку размера скачанных файлов перед прямой установкой (без менеджера пакетов), чтобы не запускать установку заведомо повреждённого файла.
5. Резервное копирование рекомендуется реализовать по схеме: регулярные wbadmín-бэкапы критичных машин + эталонный WIM-образ для быстрого восстановления аудитории.